

Exercício Prático: API de Catálogo de Livros

Turma: LionsDev **Tópicos:** APIs REST com Express.js, CRUD, JSON, arrays de objetos, parâmetros de rota, filtros por query params e status codes HTTP.

1. Contexto

Uma biblioteca comunitária quer organizar seu catálogo digital de livros. Para começar, a equipe precisa de uma API simples que permita cadastrar livros, listar todos os registros, atualizar informações, remover livros e buscar por filtros.

Neste exercício, os dados ficarão apenas em memória, dentro de um array de objetos. Quando o servidor for reiniciado, os dados cadastrados serão perdidos.

2. Configuração Inicial

Crie uma API Node.js com Express.

Requisitos iniciais:

- O arquivo principal deve se chamar `index.js`.
- A API deve rodar na porta `3000`.
- Use `express.json()` para receber dados em JSON.
- Crie um array chamado `livros`.
- Crie uma variável `proximoId` iniciando em `1`.

Cada livro deve seguir esta estrutura:

```
{
  id: 1,
  titulo: "O Hobbit",
  autor: "J. R. R. Tolkien",
  ano: 1937,
  genero: "Fantasia"
}
```

3. Requisitos Obrigatórios

3.1 Cadastrar livro (CREATE)

Crie a rota **POST /livros**.

O corpo da requisição deve receber:

```
{
  "titulo": "O Hobbit",
  "autor": "J. R. R. Tolkien",
  "ano": 1937,
  "genero": "Fantasia"
}
```

Regras:

- Todos os campos (**titulo** , **autor** , **ano** , **genero**) são obrigatórios.
- Se algum campo estiver faltando, responda com status **400** e uma mensagem clara.
- O **id** deve ser gerado automaticamente.
- O novo livro deve ser adicionado ao array **livros**.
- Ao cadastrar com sucesso, responda com status **201** e retorne o livro criado.

3.2 Listar livros (READ)

Crie a rota **GET /livros**.

Regras:

- Retorne todos os livros cadastrados.
- Se nenhum livro tiver sido cadastrado, retorne um array vazio **[]**.
- Use status **200**.

3.3 Atualizar livro (UPDATE)

Crie a rota **PUT /livros/:id**.

O corpo da requisição pode receber apenas os campos que devem ser alterados:

```
{
  "genero": "Aventura",
  "ano": 1938
}
```

Regras:

- Busque o livro pelo `id` recebido na rota.
- Converta o `id` para número antes de comparar.
- Se o livro não existir, responda com status `404` e uma mensagem de erro.
- Atualize apenas os campos enviados: `titulo`, `autor`, `ano` ou `genero`.
- Retorne o livro atualizado com status `200`.

3.4 Deletar livro (DELETE)

Crie a rota `DELETE /livros/:id`.

Regras:

- Use `.findIndex()` para localizar o índice do livro.
- Se o livro não existir, responda com status `404`.
- Use `.splice()` para remover o livro do array.
- Retorne uma mensagem de sucesso com status `200`.

3.5 Buscar livros (QUERY PARAMS)

Crie a rota `GET /livros/busca`.

A busca deve aceitar os seguintes filtros opcionais:

- `titulo`
- `autor`
- `ano`
- `genero`

Exemplos de URLs:

```
http://localhost:3000/livros/busca?titulo=hobbit
http://localhost:3000/livros/busca?autor=tolkien
http://localhost:3000/livros/busca?ano=1937
http://localhost:3000/livros/busca?genero=fantasia
```

Regras:

- `titulo`, `autor` e `genero` devem aceitar parte do texto e ignorar letras maiúsculas/minúsculas.

- `ano` deve ser comparado como número.
 - Se mais de um filtro for enviado, aplique todos os filtros em sequência.
 - Retorne os livros encontrados com status `200`.
-

4. Testes Esperados

Teste a API no Postman.

Sugestões de testes:

1. Cadastre pelo menos 3 livros de gêneros diferentes.
 2. Liste todos os livros.
 3. Atualize apenas o gênero de um livro.
 4. Atualize apenas o ano de um livro.
 5. Tente atualizar um livro inexistente.
 6. Delete um livro existente.
 7. Busque livros por título, autor, ano e gênero.
-

5. Dicas para a Implementação

- Use `req.body` para acessar os dados enviados em `POST` e `PUT`.
 - Use `req.params.id` para acessar o ID recebido na URL.
 - Use `req.query` para acessar filtros de busca.
 - Lembre-se de converter `id` e `ano` quando precisar comparar números.
 - Para filtros de texto, use `.toLowerCase()` e `.includes()`.
 - A API não precisa usar banco de dados neste momento.
-

LionsDev • Professor Nicolas Cardoso Motta

Exercício Prático de APIs REST com Express.js - Módulo 07